FLUTTER COURSE · SESSION 8 OF 15

Firebase & Authentication

Until now every app lived on one phone. Real apps know who you are, from any phone. Today you connect your app to Firebase, Google's ready-made backend, and let people create an account, log in, log out, and stay logged in after the app closes.

BY THE END OF TODAY YOU CAN

Connect a Flutter app to Firebase, build register, login and logout screens, show a clear message for every kind of error, and keep the user logged in across restarts.

The big picture: the office building

A **backend** is a computer on the internet that stores accounts and data for your app. **Firebase** is a backend you don't have to build or run: you switch features on in a website. Today we use **Firebase Authentication**. Next session, **Firestore** (a database).

IN THE BUILDING	IN FIREBASE
The building	Your **Firebase project**
The reception desk	**Firebase Auth**
Registering at reception gives you an ID badge	`createUserWithEmailAndPassword` gives you a `User`
Showing your badge to get in	`signInWithEmailAndPassword`
Handing the badge back	`signOut()`
Your badge stays in your pocket when you go home	Firebase saves the login on the device, so you stay logged in after a restart
A bell that rings whenever your badge status changes	`authStateChanges()` : a Stream that says “logged in as X” or “logged out”
Reception says “wrong password” or “that email already has a badge”	Error codes you turn into friendly messages

Setup: connecting your app to Firebase

You do this once per project. It's the least exciting part, but everything else depends on it.

1. **Create a Firebase project** at `console.firebase.google.com` → Add project. The free "Spark" plan is enough.

2. **Turn on email login:** Build → Authentication → Get started → Sign-in method → **Email/Password** → Enable → Save.

3. **Install the tools** (once per computer):

```
npm install -g firebase-tools        # the Firebase CLI (needs Node.js)
firebase login                       # opens the browser to sign in
dart pub global activate flutterfire_cli
```

   If `flutterfire` isn't found, add `$HOME/.pub-cache/bin` to your PATH.

4. **Connect the app.** In your project folder run `flutterfire configure`, pick your Firebase project, and tick Android, iOS and Web. It creates `lib/firebase_options.dart`, puts `google-services.json` in `android/app/`, and puts `GoogleService-Info.plist` in `ios/Runner/` for you.

5. **Add the packages:** `flutter pub add firebase_core firebase_auth`

6. **Start Firebase in `main()`**, before the app runs. It's the same async `main` pattern as Session 7:

```dart
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options:
DefaultFirebaseOptions.currentPlatform);
  runApp(const ProviderScope(child: MyApp()));
}
```

IS IT SAFE TO PUT FIREBASE_OPTIONS.DART ON GITHUB?

Yes. Those values are like a shop's **address**, not its key: they say which Firebase project to talk to, and every app shipped to users contains them. What protects your data is Firebase's **security rules**, which is Session 9's topic.

Firebase words, in plain English

CODE	WHAT IT MEANS
`FirebaseAuth.instance`	The reception desk. Every auth call starts here.
`User`	A logged-in person. Has `uid` (a unique id, used in Session 9) and `email`.
`User?`	A User means **logged in**. `null` means **logged out**.
`currentUser`	Who's logged in right now (or null).
`Stream`	Like a Future that can answer **many times**. A Future answers once; a Stream keeps sending new values.
`authStateChanges()`	A `Stream<User?>` that sends a new value at app start, after login and after logout.
`StreamProvider`	The Riverpod way to watch a Stream. Gives an `AsyncValue`, just like `FutureProvider` in Session 6.
`FirebaseAuthException`	The error Firebase throws. Its `code` says what went wrong.

The concepts, in the order we cover them

1 Register, log in, log out

Three calls do the real work. Everything around them is Session 4 (a validated form) and Session 6 (`async`, a loading state, errors).

```
final auth = FirebaseAuth.instance;

await auth.createUserWithEmailAndPassword(email: email, password: password);   //
register
await auth.signInWithEmailAndPassword(email: email, password: password);       //
log in
await auth.signOut();                                                           //
log out
```

After registering, open the Firebase console → Authentication → **Users**. Your new account is there.

---

2 Staying logged in: authStateChanges and the auth gate

Firebase remembers the login on the device. `authStateChanges()` tells your app who is logged in, every time it changes. A small widget called an **auth gate** decides which screen to show:

`loading`	`user == null`	`user != null`
Checking the saved login. Show a spinner.	Nobody is logged in. Show the `Login` screen.	Someone is logged in. Show the `Home` screen.

```
final authStateProvider = StreamProvider<User?>((ref) {
  return FirebaseAuth.instance.authStateChanges();
});

class AuthGate extends ConsumerWidget {
  const AuthGate({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return ref.watch(authStateProvider).when(
      loading: () => const Scaffold(body: Center(child:
CircularProgressIndicator())),
      error: (error, _) => Scaffold(body: Center(child: Text('$error'))),
      data: (user) => user == null ? const LoginScreen() : const HomeScreen(),
    );
  }
}
```

- Use `AuthGate` as the app's `home`.

- **After `signOut()` you don't navigate anywhere.** The stream sends `null`, and the gate swaps to the Login screen by itself. Logging in works the same way in reverse.
- **The restart test:** log in, close the app completely, and open it again. You land straight on Home.

---

3 The right message for every error

Firebase throws a `FirebaseAuthException`. Turn its `code` into a sentence, just like Session 6's `ApiException`:

```
try {
  await FirebaseAuth.instance.signInWithEmailAndPassword(email: email, password:
password);
} on FirebaseAuthException catch (e) {
  setState(() => _error = messageFor(e.code));    // show it under the form
}

String messageFor(String code) {
  switch (code) {
    case 'email-already-in-use':   return 'An account already exists for this
email. Try logging in.';
    case 'weak-password':          return 'Password must be at least 6
characters.';
    case 'invalid-email':          return 'Enter a valid email address.';
    case 'invalid-credential':
    case 'wrong-password':
    case 'user-not-found':         return 'Email or password is incorrect.';
    case 'network-request-failed': return 'No internet connection. Check your Wi-Fi
or mobile data.';
    case 'too-many-requests':      return 'Too many attempts. Please wait a few
minutes and try again.';
    case 'user-disabled':          return 'This account has been disabled.';
    default:                       return 'Something went wrong. Please try
again.';
  }
}
```

CODE	WHEN IT HAPPENS
`email-already-in-use`	Registering with an email that already has an account
`weak-password`	Password shorter than 6 characters
`invalid-email`	The email isn't a real email format
`invalid-credential`	Wrong email or wrong password (see the note below)
`network-request-failed`	No internet
`too-many-requests`	Too many failed attempts in a short time
`user-disabled`	The account was switched off in the console

WHY NOT “WRONG-PASSWORD”?

New Firebase projects switch on **email enumeration protection**. Firebase then won't say whether the email or the password was wrong, and returns `invalid-credential`

4

Showing who is logged in

```
final user = ref.watch(authStateProvider).value;     // User? : null while loading or logged out
Text('Logged in as ${user?.email}');

FilledButton(
  onPressed: () => FirebaseAuth.instance.signOut(),   // the gate shows Login by itself
  child: const Text('Log out'),
)
```

Errors you will see today

> “No Firebase App '[DEFAULT]' has been created”
>
> `Firebase.initializeApp` is missing from `main()`, or isn't awaited.

> `operation-not-allowed`
>
> Email/Password sign-in isn't switched on. Firebase console → Authentication → Sign-in method → Email/Password → Enable.

> `"flutterfire: command not found"`
>
> Add `$HOME/.pub-cache/bin` to your PATH, then open a new terminal.

> `In Chrome, you're logged out after you stop and run the app again`
>
> Same as Session 7: Chrome keeps the login per address, and Flutter picks a new random port each run. Run on a fixed port: `flutter run -d chrome --web-port 5050`. Phones don't have this problem.

> `The spinner never stops after Log in`
>
> You forgot to set `_loading = false` in the error branch, so a failed login leaves the button spinning.

Practice task (in class)

- [ ] Create a Firebase project, enable Email/Password, and run `flutterfire configure` in a new Flutter project.
- [ ] Build a **Register** screen and a **Login** screen with validated email and password fields.
- [ ] Show a different, clear message for every error in the table above. Try each one on purpose.
- [ ] Add an auth gate, so a logged-in user goes straight to Home.
- [ ] Add **Log out** on the Home screen. Check you're still logged in after closing and reopening the app.

Project: Login Starter App

Give the **Paws & Homes** login screen from the design you already have a real backend.

- [ ] Login and Register screens that match the Paws & Homes design: colours, fonts, field styles and the loading button.
- [ ] Register with name, email, password and confirm password. All fields validated.
- [ ] A distinct error message for every failure case, shown under the form.
- [ ] An auth gate using a Riverpod `StreamProvider`. The session survives a restart.

- [ ] Home screen shows “Hello, *email*” and a Log out button.
- [ ] Bonus: a “Forgot password?” link using `sendPasswordResetEmail(email: ...)`.
- [ ] Commit and push to your own GitHub repository.

Check yourself

▼ What is a backend, and why do apps need one?

Computers on the internet that store accounts and data. Without one, data lives on a single phone and is lost when the phone is.

▼ What's the difference between a Future and a Stream?

A Future gives one answer, once. A Stream can keep giving new answers over time.

▼ What does authStateChanges() send after you log out?

`null`, which means nobody is logged in. The auth gate then shows the Login screen.

▼ Why don't we navigate to the Login screen after signOut()?

The auth gate watches `authStateChanges()`. When it sends null, the gate swaps to Login by itself.

▼ Why does a wrong password give invalid-credential instead of wrong-password?

Email enumeration protection hides whether the email or the password was wrong, so attackers can't discover which emails have accounts.

NEXT SESSION

Session 9 · Firestore Database. Saving data in the cloud, tied to the logged-in user, updating live on every device, and security rules so one user can never read another user's data.